

What I learned is:

While studying the register-level driver code, the basic structure and direct access to peripheral registers were relatively easy to follow. The difficult parts were understanding register addresses, bit manipulation, peripheral clock configuration, status flags, and the sequence of operations required for interrupt-driven communication. I learned that embedded driver design requires precise hardware knowledge, clear separation between initialization, data transfer, and interrupt-handling functions, and careful management of flags and registers. Direct register programming also provides better control and performance, but it requires more attention to the microcontroller reference manual and hardware timing requirements